

Settlers of Catan Model, Type 2 project.

TRAVIS MINOR and JOSHUA VOLCHYOK

1 Abstract

This paper proposes a reinforcement learning (RL) based approach for playing the game *Settlers of Catan* while leveraging a graph neural network (GNN) based game state representation. The Catan game environment offers significant challenges to this, including: stochasticity (dice rolls), partially observable board states, and the complex spatial dynamics linked to actions. To address these, we've chosen to model the game board as a heterogeneous graph and integrate it into a proximal policy optimization (PPO) framework using an actor-critic architecture.

A curriculum learning strategy and shaped reward function have been utilized in order to continually guide the agent across increasingly difficult opponents, starting with basic heuristic and search based agents. The experimental results show that the agent is able to achieve a 50% win rate against an alpha-beta agent and others. However, performance required substantial training time, with the agent converging on that point only after approximately 40 million environment steps.

The results we have achieved with our model demonstrates the effectiveness of combining reinforcement learning with graph-based representations for structured decision-making problems, while also highlighting the challenges lying within sample efficiency and scalability.

2 Introduction

The objective of this project is to develop a reinforcement learning agent capable of playing the *Settlers of Catan* board game using a graph neural network (GNNs) within the Gymnasium framework. Catan presents a challenging environment for RL for many reasons; stochasticity with dice rolls, partial observability, long-term vs short-term planning/resource management, and a structured board. Traditionally, RL agents applied to board games excel when the game allows for a more perfect information environment, such as with the case of AlphaGo (Silver 2016).

This project proposes to solve this issue by encoding the Catan board state in a dynamic graph and train an RL agent over this representation to develop effective policies for settlement and resource management. By utilizing this approach, we can better integrate the GNN into the policy and value network, this allows for an attempt at bridging advances in geometric deep learning with a challenging real-world inspired game environment. Potentially influencing applications in network optimization, resource management, and multi-agent coordination.

3 Related Works

Recent advancements in reinforcement learning and graph neural networks have enabled amazing breakthroughs in game playing agents and graph structured reasoning. With, classic game based RL works such as DQN (Minh 2015), and AlphaGo/AlphaZero (Silver

2016; Silver 2018) have allowed machines to excel at critical thinking. And with GNN research (Scarselli 2009; Kipf 2017; Veličković 2018; Battaglia 2018), having established methods for learning on the less traditional graph structured based data. While more combine both RL and GNNs to improve upon generalizations (Almasan 2019; Gao 2019; Munikoti 2022), but these, however, focus on constructed problems. However, in the case of a *Settlers of Catan*, the agent must be able to bridge these areas; It requires RL for strategizing under uncertainty, and a GNN to understand the boards spatial relationships.

Deep Q-Networks (Mnih 2015) introduced CNN-based Q-learning to handle high-dimensional inputs, allowing the agent to produce human level gameplay with the Atari game console. Over time policy-gradient methods have allowed for improvements in stability of agents; PPO (Schulman 2017) uses a clipped surrogate objective for efficient updates, and Soft Actor-Critic (Haarnoja 2018) adds an entropy term for exploration in continuous domains. While these methods usually form the core of modern RL approaches they, however, typically assume fully observable states and single agent environments. Due to these assumptions, the usual methods will struggle to fully comprehend the Catan environment.

Deep RL has continually been improved to tackle exponentially complex games. AlphaGo (Silver 2016) as the starting point leveraged deep policy and Monte Carlo Tree Search to defeat human Go champions. Along the same path AlphaZero (Silver 2018) was able to generalize this via tabula-rasa self-play to master Go, Chess and Shogi. In real time strategy games AlphaStar (Vinyals 2019) was able to train an entire league of agents on *Starcraft II*. These agents were able to reach the highest competitive rank of "Grandmaster" within *Starcraft II*. Similarly OpenAI Five (Berner 2019) applied massive self-play to *Dota 2*, a MOBA. This approach was uniquely able to handle long term planning, hidden information, resource management, and the second to second decisions required to compete at a high enough level to beat world champion teams. All of the results for each of these games demonstrate the potential for RL within a Catan environment, but notable limitations still remain, and Catans unique environment still provides challenges.

Graphical neural networks allow deep learning to understand relational data more. The original GNN was brought from message passing but was optimized for deep architectures (Scarselli 2009). Proposed after this was the Graph Convolutional Network by Kipf and Welling (2017) allowing for linear scaling with edges. The next evolution was the Graph Attention Network (Veličković 2018) adding methods to dynamically weight neighbor influences. Next, Battaglia et al. (2018) proposed a unified framework of GNNs for better relational deep learning. All of these works prove that the GNN can effectively encode structured relationships for the purpose of neural policies. However, due to the problems they sought to answer they neglect the nuances of a game such as Catan.

The integration of GNNs into RL is still an emerging field. However, this does not mean its without use, as shown by Almasan et al. (2019) demonstrating that a GNN embedded RL agent was able to

outperform traditional flat networks. Somewhat similarly, Gao et al. (2019) showed RL’s usage in exploring GNN architecture, highlighting potential RL+GNN synergy. Further supporting this, Munikoti et al. (2022) did survey this relationship, further validating potential synergy in future applications. All of these papers, however, do not take into account the applications of a game combining these two fields.

4 Data set

Settlers of Catan is a 2-4 player turn based board game. The board consists of 19 hexagonal tiles of five types; brick, wood, ore, wheat, sheep, and additionally one desert tile. Each tile at the start of the game is assigned a number two through 12. At the start of each turn the current player will roll 2 dice, with the number landed producing the tiles type for each of the settlements or cities touching that tile. The player then may choose to spend their resources in a number of ways, choosing to build roads, settlements or cities, or to purchase development cards. Victory points are awarded for various actions taken during the game play, including 1 for building a settlement or 2 for building a city, as well as the potential victory point development card and completing various objectives during the game. The first player to reach 10 victory points wins the game.

Due to the nature of a Reinforcement Learning based approach for the agent, the data set being trained on is naturally the game play it itself collects. As such, most of the data manipulation and prep comes from properly representing the game state for the RL and GNN models to train off of.

In order to do this, we have chosen to model *Settlers of Catan* as a stochastic, partially observable, single agent Markov decision process (MDP), utilizing Catanatron (Collazo, 2021) as our base. The game environment consists of a hexagonal tiled fixed topology board, consisting of 19 total tiles, 72 edges where roads can be placed, and 54 intersections where settlements and cities can be placed. The game loop is driven by agent actions and stochastic dice rolls, leaving us with non-deterministic state transitions.

At each step (turn) of the game, an agent observes the state of the board, chooses an action to take, and receives a reward based on the outcome of that action. Throughout this process, a sequence of transitions is generated over time. These transitions are collected as trajectories generated throughout self play and matches contain sequences of the states, actions, and rewards collected over full game episodes.

The action space is very large spanning over every single buildable area, which also depends on the current state of the board. There are 327 total possible actions. In order to properly encode this we used the method of one hot encoding to most effectively communicate the state to the model. As the game progresses the action space’s mask of possible actions will shrink as nodes and edges get filled up with cities, settlements and roads. This raises additional challenges for the agent as its action space will change overtime as the game progresses and the board develops.

Overall this environment offers a unique challenge to traditional RL algorithms and strategies, directly opposing the optimal environment. With our approach we sought to minimize this innate friction of the environment through the methods we have outlined.

5 Approach

Our approach was a Graph Neural Network and a Proximal Policy Optimization (PPO) algorithm utilizing an actor critic model. The model is being trained using a shaped reward algorithm and a curriculum that increases in difficulty over time.

Firstly was the graph representation. The board is represented using PyTorch Geometric heterogeneous graph data type. This allows relationships between tiles, roads, resources, and other elements of the game to be more cleanly represented. The types of nodes in our graph representation are tiles and intersections. We can designate specific features to these node types. For tiles, we have the one-hot encoded resource of the tile, with 6 slots. One for each resource plus the desert/robber tile. They additionally have the probability of the resource to be rolled, which is static, and whether the robber is there or not. As for the second type of node, is the intersections. The intersections have the settlement ownership one-hot encoded, city ownership one-hot encoded, what kind of port and if the port is present, production value (the computed likelihood of surrounding resources being produced), and whether the node can be settled on. For edges, there are three different kinds. Tile $\rightarrow$ intersection, which represent for which nodes the tile produces for, intersection $\rightarrow$ tile, which represents what tiles the node borders, and intersection $\rightarrow$ intersection.

The purpose of this approach is to have much more detailed and nuanced relationships to be understood by our model. We are able to represent features such as road networks, which matter for blocking other players and gaining advantages such as longest road, or settlement locations and strategies, which heavily depend on adjacency, long term changes, connections between tiles and intersections, and other deep strategy that would be difficult to encode in a normal graph.

Secondly is the training method, PPO or proximal policy optimization. The PPO uses an actor critic model, which has a policy head and a value head. The policy head chooses the action for our model to take, while the value head determines the expected return based on the board state. The PPO model is robust, in that the new iterations stay close to previous. This means a lucky streak wont massively skew the training. It plays various AI and gradually moves towards more favorable outcomes.

The shaped reward function allows for shaping during non terminal steps, so it can learn between games. The initial idea was just to reward on win and punish on loss. However, this doesn’t provide any real input and significantly increases training time, as well as mostly forcing the model to make random choices without direction. So, while winning or losing a game provides by far the largest amount of reward or punishment, we provide some feedback during the game as well. This is done with victory points, playing certain development cards, and other features of the game such as longest road or largest army. The specific values are ± 10 for the outcome of a game, $\pm .05$ for the gain or loss of a victory point, and $+.01$ for playing knights.

This is performed over the course of a training curriculum. Previously, after a certain amount of steps, it transitioned to different opponents. The curriculum followed for our preliminary model is 7 phases. The phases were as follows:

- (1) Phase 0: Model vs 3 Weighted Random. 0-2,000,000 Steps.
- (2) Phase 1: Model vs Self Play (Model) vs 2 Weighted Random. 2-6,000,000 Steps.
- (3) Phase 2: Model vs 2 Self Play vs Weighted Random. 6-10,000,000 Steps.
- (4) Phase 3: Model vs Victory Point vs Self Play vs Weighted Random. 10-13,000,000 Steps.
- (5) Phase 4: Model vs Victory Point vs 2 Self Play. 13-17,000,000 Steps.
- (6) Phase 5: Model vs Alpha-Beta(Depth=2) vs Victory Point vs Self Play. 17-21,000,000 Steps.
- (7) Phase 6: Model vs Alpha-Beta(Depth=2) vs MCTS(simulations=10) vs Self Play. 21-25,000,000 Steps.

Additionally, entropy was introduced to encourage exploration. The entropy decreases as training goes on, to reduce deviation from working strategies as opponents get more difficult.

This has now changed to being win rate based, where maintaining an average win rate against the current phase of opponents of 50% or more results in the model progressing to the next stage. It has proven more effective at controlling the pace of the models training, as will be shown in the results.

Our baseline is existing competitors that are already being used for training, those being the Weighted Random, The Alpha-Beta search, and the Monte Carlo Tree Search. Their win rates against each other provide a good baseline for the effectiveness of our model (Collazo, 2021). Table 1 shows the win rates of various agents in 1v1 matchups.

Player	Mean	Num games
AlphaBeta(n=2)	80% vs ValueFunction	25
ValueFunction	90% vs GreedyPlayouts(n=25)	25
GreedyPlayouts(n=25)	100% vs MCTS(n=100)	25
MCTS(n=100)	60% vs WeightedRandom	15
WeightedRandom	60% vs Random	1000
VictoryPoint	50% vs VictoryPoint	
Random	60% vs Random	1000
	—	—

Table 1. Win rates of agents in 1v1 games

6 Results

Results were collected by logging the win-rates, average returns, and entropy, and variance. The best sign of progress within our model was average return and win rate increasing steadily through each phase. Even against more difficult opponents, this trend maintained. However slowly, the model was consistent in making progress.

The initial round of training was roughly 28 million steps. The curriculum was done by step count, rather than any measured metric. The latest iteration is roughly 37.6 million steps. This may seem like a small increase, considering that is less than ten million more steps. This is unfortunately the case as models like Alpha Beta Search and a Monte Carlo Tree search take significantly more intense levels of processing power and time in order to return proper results. Past

phase five, results and progress rapidly slowed. Within those ten million steps, average return increased from -9.0 to -4.0. Winrate increased from 5% to 50%.

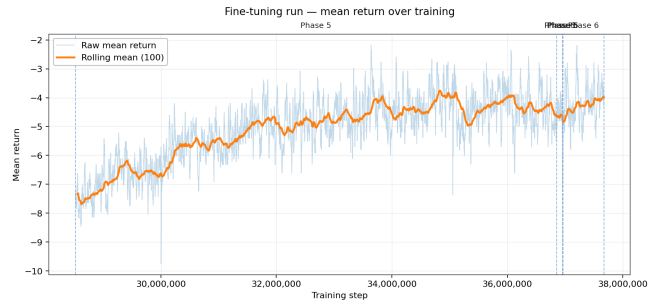


Fig. 1. Mean return through phase 5 and phase 6 of training.

Shown above is the winrate from the end of the preliminary iteration to the current phase of training. We can see the steady, but increasingly plateauing growth of the average return. By retaining the model longer in phase 5, we allow the win rate to grow fully before moving on to the next stage. This allows for a much stronger, fine tuned mode that can learn more about fundamentals of the game before moving onto advanced opponents.

If we compare this to our preliminary results, we can see that the increase reaches a marginally higher peak, at the cost of a reduced growth rate. Since the goal is to produce a more powerful opponent, as opposed to a faster training one, this is a success.

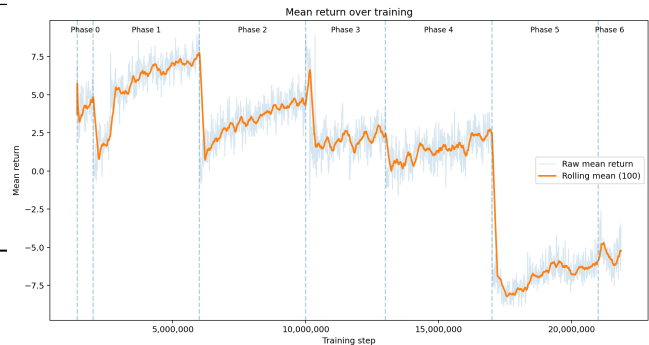


Fig. 2. Mean return in each phase of training in preliminary results.

The figure above are the preliminary results average return. The return in phase five is lower at its peak. A glaring advantage of allowing the training in phase 5 to continue for longer is transition between phase 5 and phase 6. When moving from phase 5 to phase 6, as seen in figure 2 the average return is maintained and even exceeds the average return of the preliminary results.

An additional interesting insight is the amount of steps per game.

Games get shorter as the AI gets better. More efficient pathes are found, less random moves are made, and overall the best state of the board is learned. We see a definite lowering once weighted

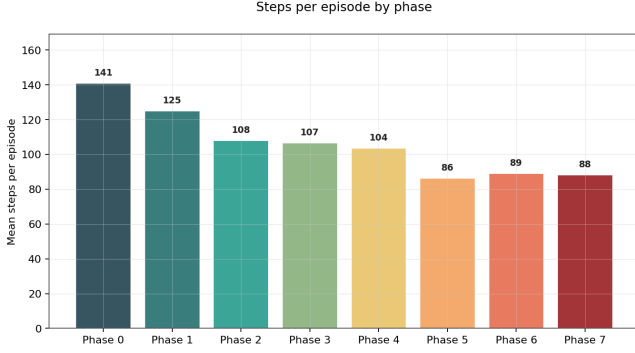


Fig. 3. The amount of steps (turns) per game taken by our GNN player.

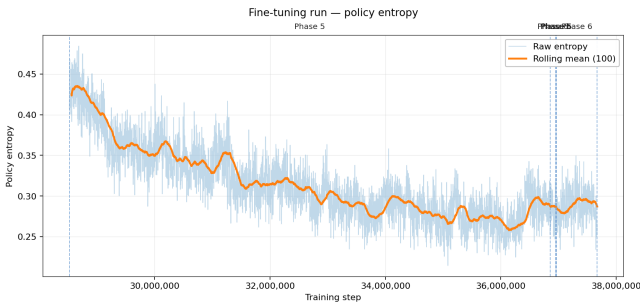


Fig. 4. Change in entropy between 28 million and 38 million steps.

random as removed from the pool of opponents. The average win rate hovered between 80 and 90. Interestingly enough, the steps surged by a few moves in the transition from phase five to phase six. This is likely a direct result of the other side of the difficulty coin. The bots have reached a level where they can counter play each other, leading to the possibility of blocking and robbing being rewarded. This leads to games becoming more competitive, and therefore longer again.

In this second round of training, we also get a glimpse of the entropy changes experienced by the model.

Entropy gives us a unique view into how "stuck" the model feels. If there is a sense of plateauing, entropy can increase to pull the model out of a local minimum. This allows more paths to be explored before settling for a correct strategy. Using the previous figures, we can infer that as the winrate went up, entropy went down due to less need to explore with more consistent wins.

Another important result is the policy loss and value loss function show how the PPO works to prevent large policy updates as shown by Figure 5. The value loss function also tells us some very interesting information, most notably the spikes at the phase transitions. These spikes show that the value function cannot accurately predict returns whenever a phase transitions to a harder learning environment. These spikes also show that the critic needed to re-learn good strategies, demonstrating the curriculum was working to enrich our agent. The slight overall downward trend shows that the critic overtime adapts to the situation.

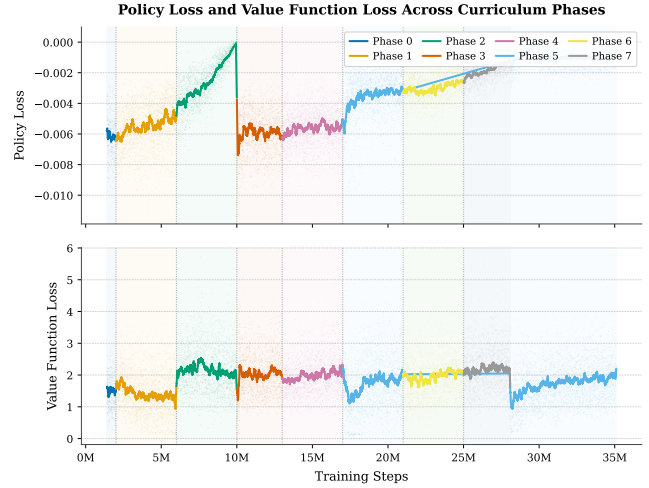


Fig. 5. Policy Loss and Value Function Loss Across Curriculum Phases

7 Conclusion

In this project, we have explored a reinforcement learning (RL) and graph neural network (GNN) solution for the *Settlers of Catan* game. By modeling the board as a graph and utilizing a GNN based encoder, we sought to capture the relational dependencies that contribute to the game's complex strategies. Approaching the problem in this way allows for a natural way to represent local and global interaction on the board. Local involving such things as settlement and road placement and global involving expansion and opponent interference.

Although the Catan environment introduces challenges such as stochasticity and spatial relations between actions, our approach demonstrates how introducing a structured representation of the data can contribute to improved learning in complex board games. In more general terms, this work seeks to highlight the potential of combining reinforcement learning with graph neural networks to improve decisions involving relational problems.

The results show that this approach is effective, achieving a win rate of 50% in phase 5 of the curriculum against an alpha-beta baseline and two additional agents. However, this approach comes at a significant computational cost, requiring approximately 35 million training steps. While these results that this approach is capable of training a high level agent, significant tuning and time is required for achieving a more skilled agent.

Several directions exist to improve the proposed approach. The first of these is to expand on the current action state for the agent, due to simplicity we have purposely disallowed trading among the agents, if we were to continue this project along with adding trading it would be beneficial to develop our own Catan environment optimized for training our agent. Further improvements may involve; implementing more expressive encoding through a more optimized GNN environment, optimizing our PPO or experimenting with other algorithms, uniquely breaking down individual "phases" of the game to train them separately (such as the opening placements).

Overall, this work demonstrates that combining reinforcement learning with GNN encoded representations is a promising direction for tackling complex and structured decision making problems.

8 References

- (1) Almasan, P., et al. 2019. Deep reinforcement learning meets graph neural networks. arXiv preprint arXiv:1910.07421.
- (2) Battaglia, P. W., et al. 2018. Relational inductive biases, deep learning, and graph networks. arXiv preprint arXiv:1806.01261.
- (3) Berner, C., et al. 2019. Dota 2 with large scale deep reinforcement learning. arXiv preprint arXiv:1912.06680.
- (4) Gao, Y., et al. 2019. GraphNAS: Graph neural architecture search with reinforcement learning. arXiv preprint arXiv:1904.09981.
- (5) Haarnoja, T., et al. 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning. ICML.
- (6) Jiang, J., et al. 2020. Graph neural networks for multi-agent reinforcement learning. ICLR.
- (7) Mnih, V., et al. 2015. Human-level control through deep reinforcement learning. Nature.
- (8) Munikoti, S., et al. 2022. Challenges and opportunities in deep reinforcement learning with graph neural networks. arXiv preprint arXiv:2206.07922.
- (9) Kipf, T. N., and Welling, M. 2017. Semi-supervised classification with graph convolutional networks. ICLR.
- (10) Silver, D., et al. 2016. Mastering the game of Go with deep neural networks and tree search. Nature.
- (11) Silver, D., et al. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science.
- (12) Scarselli, F., et al. 2009. The graph neural network model. IEEE Transactions on Neural Networks.
- (13) Schulman, J., et al. 2017. Proximal policy optimization algorithms. ICML.
- (14) Vinyals, O., et al. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. Nature.
- (15) Veličković, P., et al. 2018. Graph attention networks. ICLR.
- (16) Almasan, P., et al. 2019. Deep reinforcement learning meets graph neural networks. arXiv preprint arXiv:1910.07421.
- (16) Collazo, B. 2021. Catanatron: Settlers of Catan bot simulator and strong AI player. GitHub repository. <https://github.com/bcollazo/catanatron>